设计原则

开闭原则(openclose)

对扩展开放，对修改关闭；减少源代码的修改，提升扩展性，一般来讲，我们会对外不暴露set方法

依赖倒置原则(dependencyinversion)

依赖倒置原则（Dependence Inversion Principle,DIP）是指设计代码结构时，高层次的模块不依赖于低层次的模块，二者都应该依赖其抽象

抽象不依赖细节，细节应该依赖抽象

单一职责原则(simpleresponsibility)

单一职责（Simple Responsibility Pinciple，SRP）是指不要存在多于一个导致类变更的原因。假设我们有一个 Class 负责两个职责，一旦发生需求变更，修改其中一个职责的逻辑代码，有可能会导致另一个职责的功能发生故障。这样一来，这个 Class 存在两个导致类变更的原因。如何解决这个问题呢？我们就要给两个职责分别用两个 Class 来实现，进行解耦。后期需求变更维护互不影响。这样的设计可以降低类的复杂度，提高类的可读性，提高系统的可维护性，降低变更引起的风险。总体来说就是一个 Class/Interface/Method 只负责一项职责。

尽量的原子化，但是我们实际情况很多时候是不符合单一职责的，因为会加大代码量！！以及让代码过于分散！

接口隔离原则（Interface Segregation Principle, ISP）是指用多个专门的接口，而不使用单一的总接口，客户端不应该依赖它不需要的接口

IAnimal 接口：

```
public interface IAnimal
{
 void eat();
 void fly();
 void swim();
}
```

bird类实现

```
public class Bird implements IAnimal
{
    @Override public void eat() {}
    @Override public void fly() {}
    @Override public void swim() {}
}
```

Dog类实现：

```
public class Dog implements IAnimal
{
    @Override public void eat() {}
    @Override public void fly() {}
    @Override public void swim() {}
}
```

可以看出，Bird 的 swim()方法可能只能空着，Dog 的 fly()方法显然不可能的。所以我们可以分别设计 IEatAnimal，IFlyAnimal 和 ISwimAnimal 接口

为什么要把IAnimal拆分成IFlyAnimal,ISwimAnimal，不拆分会有什么样的问题

答：其实还是为了一个接口只干一件或者同一类事情，避免万能接口的现象，和单一职责原则可以结合起来理解，动物接口拆分是因为动物也分海陆空三种类型的动物，每一种类型的动物有不一样的特征，如果放一起，岂不是说这个动物具备海陆空三种动物的特性，逻辑上也说不过去。比如狗是不能飞的所以不能继承IFlyAnimal接口。

接口隔离原则

各个接口不要相互污染

迪米特法则(lawofdemeter)

迪米特原则（Law of Demeter LoD）是指一个对象应该对其他对象保持最少的了解，又叫最少知道原则（Least Knowledge Principle,LKP），尽量降低类与类之间的耦合。迪米特原则主要强调只和朋友交流，不和陌生人说话。出现在成员变量、方法的输入、输出参数中的类都可以称之为成员朋友类， 而出现在方法体内部的类不属于朋友类。

里式替换原则(liskovsubstitution)

原则是，子类和父类的替换不应该影响功能

即基类随便怎么改动子类都不受此影响，那么基类才能真正被复用

里氏替换原则（Liskov Substitution Principle,LSP）

1、子类可以实现父类的抽象方法，但不能覆盖父类的非抽象方法。

2、子类中可以增加自己特有的方法。

3、当子类的方法重载父类的方法时，方法的前置条件（即方法的输入/入参）要比父类方法的输入参数更宽松。

4、当子类的方法实现父类的方法时（重写/重载或实现抽象方法），方法的后置条件（即方法的输出/返回值）要比父类更严格或相等。

里氏替换原则有至少以下两种含义：

1. 里氏替换原则是针对继承而言的，如果继承是为了实现代码重用，也就是为了共享方法，那么共享的父类方法就应该保持不变，不能被子类重新定义
   。子类只能通过新添加方法来扩展功能，父类和子类都可以实例化，而子类继承的方法和父类是一样的，父类调用方法的地方，子类也可以调用同一个继承得来的，逻辑和父类一致的方法，这时用子类对象将父类对象替换掉时，当然逻辑一致，相安无事。

2. 如果继承的目的是为了多态，而多态的前提就是子类覆盖并重新定义父类的方法，为了符合LSP，我们应该将父类定义为抽象类，并定义抽象方法，让子类重新定义这些方法，当父类是抽象类时，父类就是不能实例化，所以也不存在可实例化的父类对象在程序里。也就不存在子类替换父类实例（根本不存在父类实例了）时逻辑不一致的可能。

   不符合LSP的最常见的情况是，父类和子类都是可实例化的非抽象类，且父类的方法被子类重新定义，这一类的实现继承会造成父类和子类间的强耦合，也就是实际上并不相关的属性和方法牵强附会在一起，不利于程序扩展和维护。

   如何符合LSP？总结一句话 —— 就是尽量不要从可实例化的父类中继承，而是要使用基于抽象类和接口的继承。

合成复用原则(compositereuse)

合成复用原则（Composite/Aggregate Reuse Principle,CARP）是指尽量使用对象组合(has-a)/ 聚合(contanis-a)，而不是继承关系(is-a)达到软件复用的目的。可以使系统更加灵活，降低类与类之间的耦合度，一个类的变化对其他类造成的影响相对较少

工厂模式

简单工厂模式

简单工厂模式是指由一个工厂对象创建出哪一种产品类的实例，属于创建型模式，何为创建型模式，就是指用于描述“怎样创建对象”，GoF 中提供了单例、原型、工厂方法、抽象工厂、建造者等 5 种创建型模式。

后续还会讲结构性与行为型

结构型：用于描述如何将类或对象按某种布局组成更大的结构，GoF 中提供了代理、适配器、桥接、装饰、外观、享元、组合等 7 种结构型模式。

行为型：用于描述类或对象之间怎样相互协作共同完成单个对象都无法单独完成的任务，以及怎样分配职责　GoF 中提供了模板方法、策略、命令、职责链、状态、观察者、中介者、迭代器、访问者、备忘录、解释器等 11 种行为型模式。

```
public class CourseFactory {

    public ICourse create(Class<? extends ICourse> clazz){
        try {
            if (null != clazz) {
                return clazz.newInstance();
            }
        }catch (Exception e){
            e.printStackTrace();
        }
        return null;
    }

}
```

有一个工厂类，来决定传入的是什么对象类型，去创建那个对象类！！

主要有3个重要的结构：

工厂（CourseFactory）、抽象类产品（ICourse）、具体产品（JavaCourse）

为什么一定要用工厂模式，能够举出使用简单工厂比我直接`new Object()`到底有哪些好处？

答：

1. 在对象创建比较复杂的时候，工厂模式可以封装一些创建bean的方式，需要什么样的对象，通过工厂类获取，不用重复造轮子，重复写构建对象的过程。降低代码耦合性，修改bean对象的创建，只在一个地方修改即可。
2. 在不确定什么时候生成对象或者生成对象的方式不确定的时候，工厂方法可以延迟对象初始化比如Spring中的ObjectFactory在循环依赖中的应用。

工厂方法模式

是指定义一个创建对象的接口，但让实现这个接口的类来决定实例化哪个类，工厂方法让类的实例化推迟到子类中进行

创建对象的接口

```
public interface ICourseFactory {
    ICourse create();
}
```

实现接口的类来实例化哪个类

```
public class JavaCourseFactory implements ICourseFactory {
    public ICourse create() {
        return new JavaCourse();
    }
}
```

调用

```
public static void main(String[] args) {

    ICourseFactory factory = new PythonCourseFactory();
    ICourse course = factory.create();
    course.record();

    factory = new JavaCourseFactory();
    course = factory.create();
    course.record();

}
```

为什么又有一个工厂方法，它解决了简单工厂的哪个问题?

答：简单工厂的扩展，每次都需要修改工厂里的代码，当扩展比较多的时候，代码比较臃肿，耦合性比较高
工厂方法，在扩展的时候，我只需要再添加新的实现即可，不影响别的。实际上工厂方法就是先定一个抽象接口，后面在基于这个抽象接口定义对应的工厂Factory类。

抽象工厂模式

抽象工厂模式是指提供一个创建一系列相关或相关依赖对象的接口，无须指定具体的类!

```
public abstract class CourseFactory {

    public void init(){
        System.out.println("初始化基础数据");
    }

    protected abstract INote createNote();

    protected abstract IVideo createVideo();

}
```

```
public class JavaCourseFactory extends CourseFactory {

    public INote createNote() {
        super.init();
        return new JavaNote();
    }

    public IVideo createVideo() {
        super.init();
        return new JavaVideo();
    }
}
```

抽象工厂缺点:

1. 规定了所有可能被创建的产品集合，扩展新的产品困难，需要修改抽象工厂接口
2. 增加了系统的抽象性和理解难度

题目

1.用工厂模式设计支付业务场景，包含跨境支付，支付宝、微信、银联支付,并画出类图

举例说出抽象工厂的应用场景，对于视频中的例子我如果要增加一个答疑的产品应该怎么修改代码，你也可以基于老师的代码来完善这个功能

答：抽象工厂适用于大量创建一系列相关类型的对象。

增加答疑产品需要在ICourseFactory中新增一个答疑方法，并增加一个答疑接口。以前的JavaCourseFactory/PytonCourseFactory具体的产品族都要修改增加答疑方法。

单例模式

单例模式 任何情况下，确保类只能有1个实例，并提供一个全局访问点 属于创建型模式

饿汉式单例 首次加载的时候创建，加上static ,类变量随着类的加载而加载，即使此类还未new过对象，这个类变量也存在，而且仅一份。

仔细分析双重检查锁单例模式中，第一个if(instance ==null)和第二个if(instance ==null)的作用是什么？为什么要加synchronize？不加volatile关键字会出现什么问题。

答：

1. 第一个if(instance ==null)判断对象如果!=null就不进入同步快（synchronized快）这样减少同步带来性能开销
2. 第二个if(instance ==null)防止前面一个线程刚好创建了instance释放了锁刚好第二个线程走到Synchronized进来了，如果这里不判断的话会导致重新创建了对象。

不加volatile会由于指令重排问题导致出现未完全初始化的对象，什么是指令重排呢？

`Object O = new Object()`

会做下面3件事情：

1). 开辟堆空间

2). 对开辟的空间进行初始化

3). 将内存空间地址赋值给instance对象

如果不加volatile 2和3顺序是可以打乱的，如果未初始化的对象赋值给了变量被业务使用了那么会造成不可预知的

饿汉式单例

在类加载的时候立即初始化，并且创建单例对象，绝对的线程安全，在线程还没出现以前就开始实例化了，不存在访问安全的问题

优点：线程安全，执行效率比较高

缺点：所有对象类加载的时候就实例化，那么可能会有大量的内存浪费，因为不确定这个对象会不会使用

```java
public class HungrySingleton {

    private static final HungrySingleton hungrySingleton = new
HungrySingleton();

    private HungrySingleton(){}

    public static HungrySingleton getInstance(){
        return  hungrySingleton;
    }
}
```

静态代码块机制

```java
public class HungryStaticSingleton
{
    private static final HungryStaticSingleton hungrySingleton;

    static {
        hungrySingleton = new HungryStaticSingleton();
    }

    private HungryStaticSingleton(){}

    public static HungryStaticSingleton getInstance(){
        return  hungrySingleton;
    }
}
```

懒汉式单例

为了解决饿汉式单例带来的内存浪费，出来了懒汉单例，只有在对象在使用的时候才会被初始化

```java
public class LazySimpleSingletion {
    private static LazySimpleSingletion instance;
    private LazySimpleSingletion(){}

    public static LazySimpleSingletion getInstance(){
        //有线程安全问题,可以使用
        if(instance == null){
            instance = new LazySimpleSingletion();
        }
        return instance;
    }
}
```

有线程安全问题,可以使用synchronized锁，让getInstance变成同步方法，但是不能出现并行访问的情况，严重的拖慢了处理性能，所以我们只锁住代码块，使用双重检查锁

```
public class LazyDoubleCheckSingleton {
    private volatile static LazyDoubleCheckSingleton instance;
    private LazyDoubleCheckSingleton(){}

    public static LazyDoubleCheckSingleton getInstance(){
        //检查是否要阻塞,如果已经创建过，就不需要再进入加锁代码块拉低性能，大大的提升性能，如果没有该检查，每次都会去竞争锁
        if (instance == null) {
            synchronized (LazyDoubleCheckSingleton.class) {
                //检查是否要重新创建实例
                //如果没有该判断，2个线程在第一个if都判断为null,那么就会创建2个实例
                if (instance == null) {
                    instance = new LazyDoubleCheckSingleton();
                }
            }
        }
        return instance;
    }
}
```

还是有一定性能问题，那么我们还可以采用静态内部类的方式

同样也是利用了类的加载机制，它与饿汉模式不同的是，它是在内部类里面去创建对象实例。这样的话，只要应用中不使用内部类，JVM就不会去加载这个单例类，也就不会创建单例对象，从而实现懒汉式的延迟加载。也就是说这种方式可以同时保证延迟加载和线程安全。

```
public class LazyStaticInnerClassSingleton {

    private LazyStaticInnerClassSingleton(){
        //解决反射破坏，因为反射可以调用私有的构造器
        if(LazyHolder.INSTANCE != null){
            throw new RuntimeException("不允许非法访问");
        }
    }

    private static LazyStaticInnerClassSingleton getInstance(){
        return LazyHolder.INSTANCE;
    }

    private static class LazyHolder{
        private static final LazyStaticInnerClassSingleton INSTANCE = new LazyStaticInnerClassSingleton();
    }

}
```

注册式单例

枚举式单例模式

```
public enum EnumSingleton {
    INSTANCE;

    private Object data;

    public Object getData() {
        return data;
    }

    public void setData(Object data) {
        this.data = data;
    }

    public static EnumSingleton getInstance(){return INSTANCE;}
}
```

优点：解决了反射与序列化破坏，但是也在类加载时将所有的对象初始化放在内存中去了，不适合大量创建单例对象的场景

容器式单例

```
public class ContainerSingleton {

    private ContainerSingleton(){}

    private static Map<String,Object> ioc = new ConcurrentHashMap<String,
Object>();

    public static Object getInstance(String className){
        Object instance = null;
        if(!ioc.containsKey(className)){
            try {
                instance = Class.forName(className).newInstance();
                ioc.put(className, instance);
            }catch (Exception e){
                e.printStackTrace();
            }
            return instance;
        }else{
            return ioc.get(className);
        }
    }

}
```

题目

1、解决容器式单例的线程安全问题。

单例模式总结

1. 饿汉式单例：在类加载就实例化，线程安全，执行效率比较高，但是不确定是否需要使用，造成内存浪费
2. 懒汉式单例：第一次调用进行初始化、线程不安全，需要通过synchronized或双重检查锁来实现线程安全
3. 反射破坏单例原理：虽然构造器设置为私有，但是可以通过设置强制访问来调用其构造函数，具体为：`c.setAccessible(true);`
4. 序列化破坏单例原理：反序列化后的对象会重新分配内存，即重新创建
5. readResolve()方法防止反序列化破坏单例原理：在反序列化调用readObject()方法中，会先反序列化一个实例，再进行判断是否定义了该方法，如果定义了该方法，则将刚才反序列化生成的对象进行覆盖。其实实际上实例化了两次，只不过新创建的对象没有被返回
6. 枚举式单例模式：枚举式单例模式，无法通过反射及反序列化来破坏单例。无法通过反射破坏单例是因为jdk底层做了限制，当发现反射调用的是枚举的构造器时，会抛出""异常；无法反序列化来破坏单例是因为反序列化时如果该Enum类已被实例化则通过类名及类对象找到该枚举类并返回，所以不会产生多实例。是实现单例模式最为优良的方式，并且《Effective Java》一书也推荐使用枚举来实现单例
7. 容器式单例模式：方便于管理众多的单例对象，但会出现线程安全问题，也会出现反射和反序列化破坏其单例的现象，不过spring中的对象管理通过该方式

代理模式（结构性）

属于结构型模式，指为其他对象提供一种代理，以控制对这个对象的访问，而代理对象在可以在客户端和目标对象之间起到中介的作用！！！也可以理解为代码增强，在原代码逻辑前后增加一些代码逻辑，而使调用者无感知

一般包含三种角色

抽象主题角色：声明真实主题与代理的共同接口方法，可以是接口或者抽象类

真实主题角色：被代理类，负责执行系统真正逻辑业务的对象

代理主题角色：代理类，持有代理类的引用，具备对代理类的代理权

静态代理

抽象主题角色代码，被代理类跟代理类都必须要做的事情 findLove

```
public interface IPerson {
    void findLove();
}
```

真实主题角色（被代理类） 这里是张三要findLove

```
public class ZhangSan implements IPerson {
    public void findLove() {
        System.out.println("儿子要求：肤白貌美大长腿");
    }

}
```

代理主题角色 （代理类），由zhangsan的爸爸zhanglaosan去帮忙找，但是要加强功能，之前跟之后要做些事情

```
public class ZhangLaosan implements IPerson {

    private ZhangSan zhangsan;

    public ZhangLaosan(ZhangSan zhangsan) {
        this.zhangsan = zhangsan;
    }
    public void findLove() {
        System.out.println("张老三开始物色");
        zhangsan.findLove();
        System.out.println("开始交往");
    }
}
```

上面就是一个典型的代理模式的实现，但是我们发现了一个问题，zhanglaosan只会去给zhangsan findLove()。别人家的孩子是不会管的！！！这个就是我们的静态代理

但是，随着社会的进步与发展，就出现了媒婆相关的中介，就是我们下面要讲的动态代理模式!

动态代理

JDK代理

采用字节重组，重新生成对象来替代原始对象，以达到动态代理的目的

代理类

```
public class JdkMeipo implements InvocationHandler {
    private IPerson target;
    public IPerson getInstance(IPerson target){
        this.target = target;
        Class<?> clazz =  target.getClass();
        return (IPerson)
Proxy.newProxyInstance(clazz.getClassLoader(),clazz.getInterfaces(),this);
    }

    public Object invoke(Object proxy, Method method, Object[] args) throws
Throwable {
        before();
        Object result = method.invoke(this.target,args);
        after();
```

```
        return result;
    }

    private void after() {
        System.out.println("双方同意，开始交往");
    }

    private void before() {
        System.out.println("我是媒婆，已经收集到你的需求，开始物色");
    }
}
```

被代理类

```
public class Zhangsan implements IPerson {

    public void findLove() {
        System.out.println("张三要求：肤白貌美大长腿");
    }

    public void buyInsure() {
        System.out.println("30万");
    }

}
```

调用：

```
public class Test {
    public static void main(String[] args) {
        JdkMeipo jdkMeipo = new JdkMeipo();
        IPerson zhangsan = jdkMeipo.getInstance(new Zhangsan());
        zhangsan.findLove();

    }
}
```

步骤如下：

1.获取被代理对象的引用，并且反射获取它所有的接口

2.jdk动态代理类重新生成一个新的类，同时新的类要实现被代理类实现的所有接口

3.动态生成java代码，根据新加的业务方法生成

4.编译新生成的class文件，重新运行

所以，JDK代理的目标对象必须要实现接口，因为JDK代理是根据被代理对象的所有接口去生成新的方法！

但是Proxy.newProxyInstance返回的是接口类，而非实现，所以JDK只能代理接口！！

CGLB代理

代理类

```
public class CGlibMeipo implements MethodInterceptor {


    public Object getInstance(Class<?> clazz) throws Exception{
        //相当于Proxy，代理的工具类
        Enhancer enhancer = new Enhancer();
        enhancer.setSuperclass(clazz);
        enhancer.setCallback(this);
        return enhancer.create();
    }

    public Object intercept(Object o, Method method, Object[] objects,
MethodProxy methodProxy) throws Throwable {
        before();
        Object obj = methodProxy.invokeSuper(o,objects);
        after();
        return obj;
    }

    private void before(){
        System.out.println("我是媒婆，我要给你找对象，现在已经确认你的需求");
    }

    private void after(){
        System.out.println("OK的话，准备办事");
    }
}
```

被代理类

```
public class Customer {

    public void findLove(){
        System.out.println("儿子要求：肤白貌美大长腿");
    }
}
```

调用

```
public class CglibTest {
    public static void main(String[] args) {
```

```
        try {


            //JDK是采用读取接口的信息
            //CGLib覆盖父类方法，对指定目标类生成一个子类，所以采用的是继承
            //目的：都是生成一个新的类，去实现增强代码逻辑的功能

            //JDK Proxy 对于用户而言，必须要有一个接口实现，目标类相对来说复杂
            //CGLib 可以代理任意一个普通的类，没有任何要求

            //CGLib 生成代理逻辑更复杂，效率,调用效率更高，生成一个包含了所有的逻辑的
FastClass，不再需要反射调用
            //JDK Proxy生成代理的逻辑简单，执行效率相对要低，每次都要反射动态调用

            //CGLib 有个坑，CGLib不能代理final的方法


 //System.setProperty(DebuggingClassWriter.DEBUG_LOCATION_PROPERTY,"D://cglib_pr
oxy_classes");

            Customer obj = (Customer) new
CGlibMeipo().getInstance(Customer.class);
            //System.out.println(obj);
            obj.findLove();
        } catch (Exception e) {
            e.printStackTrace();
        }

    }
}
```

步骤

1. 创建被代理的类及方法；
2. 创建一个实现接口 MethodInterceptor 的代理类，重写 intercept 方法；
3. 创建获取被代理类的方法 getInstance(Object target);
4. 获取代理类，调用代理类方法。

CGLib代理的不需要被代理对象实现任何接口，它会重写被代理类的所有方法，所以不能代理final修饰的方法或者类

因为被final修饰后，表明这个类不能被继承

JDK动态代理跟CGLib动态代理的区别 (面试高频)

1. JDK动态代理通过反射去实现被代理对象的所有接口，所以JDK只能代理实现了接口的对象，CGLlib对代理类所有方法重写
2. JDK动态代理跟CGLib代理都是在运行期生成新的字节码，但是CGLib实现更为复杂，用的是ASM框架，生成代理类的效率比JDK低

3. JDK动态代理是通过反射机制去查询所有被代理对象的接口，CGLib代理是通过FastClass机制直接调用方法，所有执行效率，CGLib比JDK代理要高

spring用的是哪个代理模式　bean有实现接口的时候用jdk，没有实现的话模式cglib

也可以强制用cglib

那么，代理可以代理存粹的接口吗？

装饰器模式（结构）

代码特点：一定有一个构造函数的参数为当前类实现的接口类参数

也称为包装模式，是指不改变原有对象的基础上，将功能附加到对象上，提供了比继承更有弹性的替代方案，属于结构型模式

装饰器模式的核心是功能扩展，主要包含四种角色

抽象组件：可以是接口或者抽象类，充当被装饰类的原始对象，规定了装饰对象的行为

具体组件：实现、继承抽象组件的一个具体对象，被装饰对象

抽象装饰器：

具体装饰器：

抽象组件

```
public abstract class Battercake {

    protected abstract String getMsg();

    protected abstract int getPrice();

}
```

具体组件：

```
public class BaseBattercake extends Battercake{

    protected String getMsg(){ return "煎饼";}

    public int getPrice(){ return 5;}

}
```

抽象装饰器

```
public class BattercakeDecorator extends Battercake{

    private Battercake battercake;

    public BattercakeDecorator(Battercake battercake) {
        this.battercake = battercake;
    }

    protected String getMsg(){ return this.battercake.getMsg();}

    public int getPrice(){ return this.battercake.getPrice();}

}
```

具体装饰器

```
public class EggDecorator extends BattercakeDecorator{

    public EggDecorator(Battercake battercake) {
        super(battercake);
    }
    protected String getMsg(){ return super.getMsg() + "1个鸡蛋";}
    public int getPrice(){ return super.getPrice() + 1;}
}
```

装饰器和代理模式对比

1. 仔细对比一下修饰着模式和代理模式结构图，想一想你怎么区分它们，从这里给你什么启发。

   答：装饰器模式和代理模式的结构图是一模一样的。装饰器模式更多的是强调功能的增强，代理模式更多的是强调帮代理类进行业务处理。这告诉我们学习设计模式一定从场景出发设计模式就是一些经验的总结在解决这个场景下怎么利用Java多态等语法写出优雅的代码使用代码扩展性更强。

2. 分析一下java IO代码 `new BufferedReader(new FileReader(""))` 为什么可以层层包装，画出类图或从代码层面描述怎么实现的

   ```
   在BufferedReader中的构造方法中有Reader类，而FileReader的父类是InputStreamReader，
   InputStreamReader父类是Reader。
   这个的结构符合装饰器模式。

   public BufferedReader(Reader in) {
           this(in, defaultCharBufferSize);
   }
   ```

委派模式（结构性）

又叫委派模式，是一种面向对象的设计模式，允许对象组合实现与继承相同的代码重用！

基本作用是负责任务的调用和分配任务，是一种特殊的静态代理，可以理解为全权代理，但是也有一定的区别，等下我们再分析下

主要有3个角色：

抽象任务角色：定义一个抽象接口，有若干实现类

委派者角色：负责在各个角色实例之间做出决策，并判断调用具体的实现方法

具体任务角色：真正执行任务的角色

抽象任务角色

```
public interface IEmployee {
    void doing(String task);
}
```

具体任务角色

```
public class EmployeeA implements IEmployee {
    protected String goodAt = "编程";
    public void doing(String task) {
        System.out.println("我是员工A，我擅长" + goodAt + ",现在开始做" +task + "工
作");
    }
}
```

委派者角色

```
public class Leader implements IEmployee {

    private Map<String,IEmployee> employee = new HashMap<String,IEmployee>();

    public Leader(){
        employee.put("爬虫",new EmployeeA());
        employee.put("海报图",new EmployeeB());
    }

    public void doing(String task) {
        if(!employee.containsKey(task)){
            System.out.println("这个任务" +task + "超出我的能力范围");
            return;
        }
        employee.get(task).doing(task);
    }
}
```

调用者

```
public class Test {
    public static void main(String[] args) {
        new Boss().command("海报图",new Leader());
        new Boss().command("爬虫",new Leader());
        new Boss().command("卖手机",new Leader());
    }
}
```

双亲委派

JVM类加载双亲委派，一个类加载器在加载类时，首先把请求委派给自己的父类加载器去执行。如果父类加载器还存在父类加载器，就继续向上委派，直到顶层的启动类加载器，如果父类的加载器能够完成类加载，就成功返回，如果父类加载无法完成加载，那么子加载器才会尝试自己去加载

类加载器的类别（不讲）

BootstrapClassLoader（启动类加载器）

ExtClassLoader （标准扩展类加载器）

AppClassLoader（系统类加载器）

CustomClassLoader（用户自定义类加载器）

优缺点：

委派能够将一个大型的任务细化，然后通过统一管理这些子任务的完成情况实现任务的跟进

缺点

增加复杂性

委派模式与代理模式的区别

1.代理模式必须要知道被代理者，就像假如这里老板下发任务，老板必须要知道这个任务是谁来执行，然后再由经理来执行，因为代理时必须要知道被代理类，比如之前代理zhangsan,但是委派模式，我不用关心是谁来执行，我只要下发命令给经理，经理给我结果就行，经理去委派任务

2.代理模式只能代理一部分，最终的结论还是得由被代理类决定，比如zhanglaosan给zhangsan找媳妇，最终还是得依靠zhangsan,看张三满不满意

3.代理模式注重过程，所以代理模式属于结构型模式，大家可以去看下静态代理，把zhangsan依赖注入到zhanglaosan,更注重的是一个结构的维护，而委派模式的目的更加注重的是结果，属于行为型模式，不关心里面的结构或者怎么去执行，重要的是提高协作效率.

委派主要解决什么问题，什么情况下要委派。

答：将不同的任务分给不同的类，让不同的类职责变得单一，当任务比较大时，可以通过委派模式将任务分发给不同的类执行。委派者不会做或者没有做该事情，而是把具体的事情交给了具体类。

策略模式（行为型）

行为型模式，面向对象的继承和多态机制，从而实现同一行为在不同场景下具备不同实现，并且结果相同

上次群里有同学讲的，我实现一个排序，可以用不同的排序算法！但是结果都是实现排序

主要包含三种角色

1. 上下文角色，用来操作策略的上下文环境
2. 抽象策略角色 ，规定策略或算法的行为
3. 具体策略角色，具体的策略或算法实现

抽象策略角色

```
public interface IPromotionStrategy {
    void doPromotion();
}
```

具体策略角色

```
public class GroupbuyStrategy implements IPromotionStrategy {
    public void doPromotion() {
        System.out.println("5人成团，可以优惠");
    }
}

public class CouponStrategy implements IPromotionStrategy {
    public void doPromotion() {
        System.out.println("使用优惠券抵扣");
    }
}
```

上下文角色

```
public class PromotionActivity {
    private IPromotionStrategy strategy;

    public PromotionActivity(IPromotionStrategy strategy) {
        this.strategy = strategy;
    }

    public void execute(){
        strategy.doPromotion();
    }
}
```

调用

```
PromotionActivity activity = new PromotionActivity(new GroupbuyStrategy());
activity.execute();
```

优点：

1.符合开闭原则

2.避免使用多重条件转移语句，如if else

3.使用策略模式可以提高算法的保密性和安全性

缺点：

1.客户端必须知道所有的策略，并且自行决定使用哪一个策略类，这个也是跟委派模式的区别所在

2.会有多个策略类，增加维护难度

委派模式跟策略模式的区别

1策略模式必须知道所有的策略，然后再决定使用哪一种策略，策略的重点在于选择哪种方式，并且是可以相互替代的，委派模式注重的是把任务下发下去

策略模式是委派模式内部的一种实现方式

自我理解：委派模式就是只有1个策略的策略模式！！

视频中的例子为什么使用策略工厂PromotionStrategyFactory

答：通过策略工厂存储所有策略，便于调用时获取正确的策略，同时避免的if...else... 的判断

策略模式使用前提是什么？即满足什么条件你才能去选策略？真正用的时候需要告诉客户端什么？当然你也可以按照约定实现策略。

答：某一种“算法”有多种实现时，可以抽象出算法的接口，所有策略实现该接口。真正使用时需要告诉用户有哪些策略，让用户选择策略，或者在用户不选择的情况下选择默认策略

观察者模式（行为型）

观察者实现方式有两种：一种是jdk实现方式，被观察者继承Observable类，观察者实现Observer接口，实现方式主要是用反射调用

一种是Guava的方式，监听者的方法的入参只能有一个，不支持自动装箱功能，所以只能是Integer

又叫发布-订阅模式，定义一种一对多的依赖关系，一个主题对象可被多个观察者对象同时监听，使得每当主题对象状态变化时，所有依赖它的对象都会收到通知并自动更新，属于行为型模式

核心是将观察者与被观察者解耦，以类似于消息、广播发送的机制联动2者

主要包含以下角色：

抽象主题、目标、发布方：被观察的对象，该角色是一个抽象类或接口，定义了增加，删除，通知观察者对象的方法

具体主题、目标、发布方：具体被观察者，当期内状态变化时，会通知已注册的观察者

抽象观察者、订阅方：定义了响应通知的更新方法

具体观察者、订阅方：在得到状态更新时，会自动做出响应

为了让大家更加容易理解，基于源代码进行了简化

事件类

```
public class Event {
    //事件触发，要通知谁（观察者）
    private EventListener target;
    //观察者给的回应
    private Method callback;

    public Event(EventListener target, Method callback) {
        this.target = target;
        this.callback = callback;
    }

    public Method getCallback() {
        return callback;
    }

    public EventListener getTarget() {
        return target;
    }
}
```

被观察者抽象

```
public class EventContext {
    protected Map<String,Event> events = new HashMap<String,Event>();

    //负责将生产者与观察者关联
    public void addLisenter(String eventType, EventListener target, Method
callback){
        events.put(eventType,new Event(target,callback));
    }


    //负责将生产者与观察者关联
    public void addLisenter(String eventType, EventListener target){
        try {
            this.addLisenter(eventType, target,
target.getClass().getMethod(eventType));
        }catch (NoSuchMethodException e){
            return;
        }
    }
```

```
    private void trigger(Event event){
        try {
            if (event.getCallback() != null) {
                //用反射调用回调函数
                event.getCallback().invoke(event.getTarget());
            }
        }catch (Exception e){
            e.printStackTrace();
        }
    }

    // 负责事件触发后的行为，通过反射调用观察者方法
    protected void trigger(String trigger){
        if(!this.events.containsKey(trigger)){return;}
        trigger(this.events.get(trigger));
    }
}
```

具体的被观察者

```
public class Mouse extends EventContext {
    public void click(){
        System.out.println("调用单击方法");
        this.trigger(MouseEventType.ON_CLICK);
    }

    public void move(){
        System.out.println("调用移动方法");
        this.trigger(MouseEventType.ON_MOVE);
    }
}
```

具体的观察者

```
public class MouseEventLisenter implements EventListener {

    public void onClick(){
        System.out.println("==========触发鼠标单击事件========\n");
    }

    public void onMove(){
        System.out.println("==========触发鼠标移动事件========\n");
    }
}
```

建造者模式（创建型）

建造者模式属于创建型模式，是将一个复杂对象的构建过程与它的表示分离，使得同样的构建过程可以创建不同的表示

建造者实用场景：

1.相同的方法，不同的执行顺序，产生不同的结果

2.多个部件或零件，装配在一个对象中，但是产生的结果又不相同

3.产品类非常复杂，或者产品类中的调用顺序不同产生不同的作用

4.当初始化一个对象特别复杂，参数多，并且很多参数都具有默认值

基本写法

```
public class CourseBuilder{

    private Course course = new Course();

    public void addName(String name) {
        course.setName(name);
    }

    public void addPPT(String ppt) {
        course.setPpt(ppt);
    }

    public void addVideo(String video) {
        course.setVideo(video);
    }

    public void addNote(String note) {
        course.setNote(note);
    }

    public void addHomework(String homework) {
        course.setHomework(homework);
    }

    public Course build() {
        return course;
    }
}
```

本来直接可以调用course赋值，但是加了层CourseBuilder，负责建造course以及调用它方法

链路写法

```
public class CourseBuilder {
    private Course course = new Course();

    public CourseBuilder addName(String name) {
        course.setName(name);
        return this;
    }

    public CourseBuilder addPPT(String ppt) {
```

```
        course.setPpt(ppt);
        return this;
    }

    public CourseBuilder addVideo(String video) {
        course.setVideo(video);
        return this;
    }

    public CourseBuilder addNote(String note) {
        course.setNote(note);
        return this;
    }

    public CourseBuilder addHomework(String homework) {
        course.setHomework(homework);
        return this;
    }

    public Course build() {
        return this.course;
    }

    @Data
    public class Course {

        private String name;
        private String ppt;
        private String video;
        private String note;

        private String homework;

        @Override
        public String toString() {
            return "CourseBuilder{" +
                    "name='" + name + '\'' +
                    ", ppt='" + ppt + '\'' +
                    ", video='" + video + '\'' +
                    ", note='" + note + '\'' +
                    ", homework='" + homework + '\'' +
                    '}';
        }
    }
}
```

链式调用

```
CourseBuilder builder = new CourseBuilder()
            .addName("设计模式")
            .addPPT("【PPT课件】")
            .addVideo("【回放视频】")
            .addNote("【课堂笔记】")
            .addHomework("【课后作业】");
System.out.println(builder.build());
```

优缺点

优点：

1. 封装性好，创建和使用分离
2. 扩展性好，建造类与产品类之间独立，一定程度上解耦

缺点

1. 产生多余的builder对象
2. 产品类发生变化后，builder也需要修改，成本较大

与工厂模式的区别

1.建造者模式更加注重方法的调用顺序，工厂模式注重于创建对象

2.创建对象的力度不同，建造者模式创建复杂的对象，由各种复杂部件组成，工厂模式创建出来的都一样

3.关注重点不一样，工厂只要创建对象，建造者还需要知道创建的对象的组成

原型模式（创建型）

原型模式，属于创建型模式，是指原型实例指定创建对象的种类，并且通过拷贝这些原型创建新的对象；核心在于拷贝原型对象，基于已有对象原型，直接基于内存二进制流进行拷贝！

浅克隆

原本在每次使用的时候都需要进行字段赋值，但是我在原型里面提供克隆接口，就会实现一劳永逸

```
@Data
public class ConcretePrototype implements Cloneable {

    private int age;
    private String name;

    @Override
    public ConcretePrototype clone() {
        ConcretePrototype concretePrototype = new ConcretePrototype();
        concretePrototype.setAge(this.age);
        concretePrototype.setName(this.name);
        return concretePrototype;
    }

    @Override
    public String toString() {
        return "ConcretePrototype{" +
                "age=" + age +
                ", name='" + name + '\'' +
                '}';
    }
}
```

但是好像还是挺麻烦的，所以JDK帮我们把整个麻烦的事情做掉了，我们只需要实现Cloneable接口即可

```
public class ConcretePrototype implements Cloneable {

    private int age;
    private String name;

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    @Override
    public ConcretePrototype clone() throws CloneNotSupportedException {
        return (ConcretePrototype)super.clone();
    }

    @Override
    public String toString() {
        return "ConcretePrototype{" +
                "age=" + age +
                ", name='" + name + '\'' +
                '}';
    }
}
```

但是当我们克隆对象里面包含引用对象，那么克隆对象与被克隆对象的引用对象发送更改，都会更改，所以，复制的只是引用内存地址，而不是值，那么这个也就是我们常说的浅克隆！！

深克隆

那么如果想深克隆，可以利用序列化实现深克隆！！

```
@Data
public class ConcretePrototype implements Cloneable,Serializable {

    private int age;
    private String name;
    private List<String> hobbies;

    @Override
    public ConcretePrototype clone() {
        try {
            return (ConcretePrototype)super.clone();
```

```
        } catch (CloneNotSupportedException e) {
            e.printStackTrace();
            return null;
        }
    }

    //浅克隆原型对象的引用对象，也可以实现引用对象分离
    public ConcretePrototype deepCloneHobbies(){
        try {
            ConcretePrototype result = (ConcretePrototype)super.clone();
            result.hobbies = (List)((ArrayList)result.hobbies).clone();
            return result;
        } catch (CloneNotSupportedException e) {
            e.printStackTrace();
            return null;
        }
    }

    public ConcretePrototype deepClone(){
        try {
            ByteArrayOutputStream bos = new ByteArrayOutputStream();
            ObjectOutputStream oos = new ObjectOutputStream(bos);
            oos.writeObject(this);

            ByteArrayInputStream bis = new
ByteArrayInputStream(bos.toByteArray());
            ObjectInputStream ois = new ObjectInputStream(bis);

            return (ConcretePrototype)ois.readObject();
        }catch (Exception e){
            e.printStackTrace();
            return null;
        }


    }

    @Override
    public String toString() {
        return "ConcretePrototype{" +
                "age=" + age +
                ", name='" + name + '\'' +
                ", hobbies=" + hobbies +
                '}';
    }
}
```

优点：

1.性能相对优良，java自带的原型模式是基于内存二进制流的拷贝

2.简化创建对象的过程

缺点：

1.加深代码的复杂度，每一个类都配置一个克隆方法

2.深克隆实现起来比较麻烦，所以，实际开发中，需要深浅得当

clone会导致单例模式失效！！

浅克隆变深克隆的几种方式?

递归clone 、序列化（Json 流）

面试题

什么是依赖倒置原则?

高层次的模块不依赖于低层次的模块，二者都应该依赖其抽象

简单的说就是面向抽象编程

双重检查锁单例模式为什么需要双重检查

第一个检查：检查是否要阻塞,如果已经创建过，就不需要再进入加锁代码块拉低性能，大大的提升性能，如果没有该检查，每次都会去竞争锁

第二个检查： 检查是否要重新创建实例。如果没有该判断，2个线程在第一个if都判断为null,那么就会创建2个实例

什么是简单工厂模式、工厂方法模式、抽象工厂模式?

简单工厂模式

主要有3个重要的角色：工厂（CourseFactory）、抽象类产品（ICourse）、具体产品（JavaCourse）

在客户端根据不同的参数来生成不同的具体产品，比如，客户端我传1，就生成java课程，传0就生成Python课程,但是主要的逻辑全部在主工厂类，后续加其他课程都需要修改工厂类，不便于维护

工厂方法模式

主要有4个重要的角色：父工厂（ICourseFactory）、具体工厂（JavaCourseFactory）、抽象类产品（ICourse）、具体产品（JavaCourse）

工厂方法与简单工厂的区别在于把创建产品的过程下移，创建产品不再由父工厂负责，而是由下面的具体工厂负责，客户端调用的时候，不再由参数决定，而是你需要什么产品，就创建什么样的工厂，比如，你需要创建Python课程，客户端为 ICourseFactory factory = new PythonCourseFactory();

抽象工厂模式

主要有4个重要的角色：抽象工厂（ICourseFactory）、具体工厂（JavaCourseFactory）、抽象类产品（ICourse）、具体产品（JavaCourse）

抽象工厂跟工厂方法的最大区别在于，我这个工厂不再是单一的创建课程，可能还会去做笔记，后续每个具体的工厂也会去做笔记！！ 不同的工厂课程做不同的笔记

什么是单例模式，并说出2到3中不同的实现方式以及他们的优缺点？

单例模式，指在任何情况下，一个类绝对只有一个实例，并提供一个全局访问点，单例模式属于创建型模式！！

实现方式：

饿汉式单例：在类加载的时候就初始化，绝对的线程安全，但是不确定类是否需要使用，会造成内存空间浪费

懒汉式单例：在使用的时候才会初始化，所以会带来线程安全问题，用synchronized或者双重检查锁解决性能问题

枚举单例：解决了反射与序列化破坏单例，反射newInstance有枚举类型判断，枚举无法通过反射创建实例！枚举的序列化则是通过类名和类对象类找到一个唯一的枚举对象，所以，也不可能被类加载加载多次！！

有哪些方式可以破坏单例模式，怎么解决单例破坏？

反射、序列化

使用枚举单例模式，不会有反射跟序列化破坏

枚举的实例的底层编译实现是一个final static class，这样的实现类似于单例中静态常量模式的实现(饿汉式)，无法实现lazy-loading但保证了单例。类在被加载时是线程安全的，所以解决了线程问题。各种序列化方法，如writeObject、readObject、readObjectNoData、writeReplace和readResolve等会造成单例破坏的方法都是被禁用的，所以在JVM中，枚举类和枚举量都是唯一的，这就实现了自由序列化

反射：无参构造函数添加非空判断

序列化：增加readResolve()方法

原型模式中的clone()

三大设计模式类型

设计模式的的3大类型其实是一个递进的过程，主要是基于它这个设计模式的目的

首先，你要实现功能，肯定要创建对象是不是，所以像单例、工厂、建造者、原型都属于怎么去很好的创建对象

创建完对象后是不是就应该考虑对象之间怎么合理的存在，如何更好的继承、依赖、组合，那么就衍生了很多结构型的模式、比如门面、适配器、代理、装饰、组合、享元

前面2步做完，就是到了具体的实现了。怎么更好的达到目的，那么为了行为更清晰，效率更高就是行为型模式，比如我们的委派，后面的策略，责任链、迭代器等等